个人报告

报告信息

项目	内容
姓名	江荣锦
学号	3126000787
专业	自动化类
考核题目	meArm 四自由度机械臂的固件与配套控制程序
报告日期	2026 年 10 月 17 日

一、对本小组其他成员考核期间表现的评价

1.1 小组分工概况

本次考核的题目是 meArm 四自由度机械臂的设计与实现。小组共 6 人，除我之外另有 5 位同学，分工大体上是"机械 + 电路 + 物资 + 软件"四条线并行：赵庆业负责机械臂组装（含夹爪），陈裕丰负责电路与接线，苏霏霏负责物资统筹与采购，沈嘉颖、莫楠参与小组的日常任务推进与代码相关部分，我本人负责机械臂固件与配套控制程序（运动学、串口协议、取放、绘图以及交付稿）。下面按成员逐一评价。

1.2 成员评价

成员	考核期间承担的工作	表现与评价
沈嘉颖	小组日常任务推进	办事认真、责任心强，常常为了把当天的进度做完而加班到深夜；交给该同学的任务不需要反复催促，是小组里最让人放心的一环。
莫楠	代码实现与优化	工作效率高，代码优化能力强，多次在功能不变的前提下把实现做得更精简，为小组节省了时间。
苏霏霏	物资统筹与采购（舵机等器件）	主动牵头小组的众筹、采购与到货跟进，把舵机、五金件等物料按时弄齐，为机械臂的装配和调试提供了及时的物料保障。
赵庆业	机械臂组装（含夹爪）	负责整机组装，其中夹爪的结构设计做得很好用，夹持可靠；后续取放与绘图两次调试都直接受益于这套机械结构。
陈裕丰	电路与接线	负责电路部分，接线与供电处理得很负责；调试期间线路问题没有成为返工的来源。

1.3 总体体会

小组整体上分工清楚、各司其职：机械和电路把实物平台搭起来，物资保障到位，软件这边才能反复上机试。我的体会是，硬件类项目里"把接口讲清楚"比"各自埋头做完"重要得多——球机结构、舵机接线和固件里的角度约定、限位窗口是同一套东西的三种表述，只要有一处没对齐，调试时就会互相怀疑。这次小组在这一点上配合得比较顺：装配、接线和固件调试基本是同步推进的，没有出现返工到彼此工作的情况。

二、个人考核部分的所学所得与感想

2.1 我在考核期间做了什么

考核期为 2026 年 10 月 2 日至 10 月 16 日，共 16 天、38 次提交、18 个版本标签（`v0.1.0` → `v1.6.7`），全部围绕 meArm 机械臂固件（Arduino UNO，C++）及其配套交付物展开：

时间	里程碑	内容要点
10-02	v0.1–v0.4 关节角控制	把摇杆从"控制末端坐标"改成直接控制四个关节角；上机实测后修正上臂 r 与下臂 c 的推杆方向；上臂限位放开到 0~180°
10-03	v0.5–v1.1 模块化与容量危机	四个物理按键（录制/播放/回中/循环取放）、A/B/C 自动取放序列、串口协议模块；三功能全开 51452 B 超出 UNO 容量，靠功能裁剪与收敛串口回复层降到 32842 B
10-04	v1.2–v1.5 协议、录制、绘图	串口改为地面坐标系、引入功能开关、录制上限 38.4 s 与回放子步插值；绘图改调公共的"移动到指定 x,y,z"核心；主 sketch 更名 `meArm.ino`
10-07	单文件整合与产物纪律	生成自包含的 `single/meArm/meArm.ino`；消除单文件内的重复宏定义；产物统一 LF、从干净 HEAD 重新生成
10-09	v1.6.4	串口回到角度语义、绘图恢复 7 个任务、摇杆手感重做两轮
10-11 ~ 10-14	文档与配置收敛	README 改写为正式项目说明（含正确的运动学模型）；删除配置化的范围限制，行程在各写入口硬编码；夹爪在夹物/绘图时每帧夹紧
10-16	v1.6.7 / v1.6.8 运动模型替换与可诊断性	运动逻辑整体替换为参照仓库最新版（θ = b−90、α = 180−r、β = c），同步重标取放点位、绘图默认值、工作空间窗口与全部文档；随后完成三项可诊断性改造与三种

		交付稿

2.2 运动学：从"自洽但错误"的模型到实物校验

这次考核我最大的收获来自一个 bug。固件里原本的映射是 `α = 90 − r`、`β = c − 180`，长度单位是 4 mm 一格，坐标 z 轴与实物方向相反。它并不是随手写的——它有完整的正解、反解、可达性判断，而且我先前为它写的一整套自检探针全部"通过"。问题是它是自洽的错误：**指令让机械臂下降，实物反而抬起**。

最后定位到根因，是把它和实物姿态逐条对：按旧模型，`x90,y0,z90`（舵机 `b=90, r=0, c=90`）应该是"上臂水平"，而实物是上臂朝上、小臂朝下。换成参照仓库的模型后，`α = 180 − r`、`β = c`、基座 `θ = b − 90`，这个姿态就自然成立了。这件事让我记住三条：

- **测试和实现同源时会一起错**。我的探针只验证了"反解再正解能回到原点"这类自洽性，没有一条与实物姿态挂钩，所以全绿并不代表正确。
- **几何结论要能落到实物上**。工作空间不是"臂长 80 + 80 = 160 mm"，而是由关节限位决定的：小臂 30°~110° 时末端到肩轴的距离只有 `d ∈ [41.4, 131.1] mm`。旧表里的取放点位在数学上"可达"、物理上够不着，最后是照真实限位重选了六个点。
- **反解的数值细节会以"路径跳变"的形式暴露**。反解在平面/肘部有多个解，只取 `+acos` 会让机械臂在两个解之间突然翻转；最后改成四个候选、并让"`+acos` 的正向/反向解优先、`−acos` 兜底"，路径才连续。另外 `RADtoDEG` 一度写成 `180/3.1415926535`（截断），以及反解行程容差取 `1e-9` 把 `r = 180` 这种零残差的解判成失败，都是靠逐点对拍才发现的。

2.3 资源受限工程：把"装不下"当成一等约束

UNO 只有 32256 B 可用 flash、2048 B SRAM，而这个项目有取放、绘图、按键三套功能。考核期间经历了三次容量危机，最深的一次三功能全开要 35162 B（107.3%），**即使用 `-mrelax` 也还有 34638 B**，装不进去。我最终的处理不是继续抠字节，而是把容量变成设计约束：

- 建立**可复现的体积账**：用 avr-gcc 7.3.0 + AVR core 1.8.8 真编，脚本支持用 `-D` 做功能消融，四个组合的体积与余量一目了然（取放+按键 22770 B、绘图+按键 29356 B、取放+绘图 31740 B / 96.9%、三开 35162 B 装不进）。
- 把"裁剪"做成**可解释的产品决定**：一块板子一次只装两个功能，默认组合改成"取放 + 绘图（按键关）"，并把这个取舍、代价和替代方案（`x90,y90,z90` 等价于 `M` 回中）写进 README。
- 新增的诊断开销也用条件编译**证明**它是零成本：把新增的上报函数整体包进 `#if WEARM_ENABLE_DRAW`，实测"取放+按键"组合一分钱没涨（22770 B 不变）。
- 我还为第二块板（按键板）单独做了真编复核：`Program 6170 B (18.8%) / Data 975 B (47.6%)`，与 arduino-cli 的历史记录一致。

2.4 可诊断性：现场只有一根串口时，要主动把判断依据打出来

考核后期的三个改动都是为了让"出问题时能查"，而不是"看起来能跑"：

- **开机横幅带上版本号**（`CMD v1.6.8 …`）。在此之前，横幅从 v1.1.0 到 v1.6.7 十来个版本都没变过，导致我无法从串口判断板子上烧的到底是哪一版——这个坑我在排查中真实踩到，一度把"板上是旧固件"误判成代码 bug。
- **把"被拒"和"真忙"分开报**：取放命令现在区分 `BUSY`（正在进行）与 `REJECTED`（参数/状态不允许）。
- **绘图开始绘制时回复带任务标签**（`D` → `OK D=TRI`），并在卡死放弃时主动上报一行 `DRW STOP t2 ph4 h2/4 x-25 y98`（任务/阶段/过点数/笔尖坐标）。以前 `D` 只回一个裸 `OK`，出问题时无法从串口判断实际画的是哪个图形、卡在哪一步。

2.5 验证体系：让"我认为对"变成"可以复核"

- **严格编译**：9 个编译单元在 `-Wall -Wextra -Wshadow -Wconversion` 下零警告。
- **与参照实现逐点对拍**：正解 6845 点与参照仓库最大偏差 `0.000000e+00 mm`；往返 3672 点最大三维误差 `6.03e-14 mm`；35301 点网格零角差；参照实现拒绝的 70 个点我也全部拒绝；14 个反解失败用例中，位姿数据都保持逐字节不变。
- **回归探针与工具链**：11 个编译探针 + 串口链路回归 + 三个绘图/取放专用工具，覆盖轴映射、协议、忙守卫、运行时开关、绘图校验、取放阶段等。
- **交付物也要被验证**：单文件交付稿不是"看起来拼好了"，而是与源码逐字校验（生成稿以源文件结尾）、单独作为一个编译单元严格编译、并用 AVR 真编确认映像大小与源码版完全一致。

2.6 踩过的坑（我想记下来的部分）

- **单文件快照不同步**：`single/meArm/meArm.ino` 是脚本生成的快照，里面有一份自己的开关副本。我改了主配置却忘了重跑生成脚本，结果"开关明明打开了绘图，命令却一直回 `REJECTED`"。后来把这个坑直接写进 README，并让 `!` 命令能读出板子上实际编进的功能位（`P=1 B=0 D=1`）。
- **凭直觉做微优化**：我为了省 flash，把整数打印从 `%10`/`/10` 改成"全程减法拆位"，结果反而多了 36 B——因为 16 位除法例程早就被别的模块链进来了，除法并不额外收费。**不确定就该实测**，这是这次最实用的一条经验。
- **测试配置写错会得到假结论**：一次漏了 `-DWEARM_BUTTON_PINS=1`，绘图工具报出 50 个"失败"，白排查了一轮。
- **工具链的编码/语法陷阱**：Windows PowerShell 5.1 读 UTF-8 无 BOM 的脚本会按本地代码页解码，中文全变乱码、脚本直接语法错误（后来给仓库加了 BOM 检查脚本）；`@('-mmcu=' + $mcu, …)` 里逗号优先级高于 `+`，会让所有编译参数被拼成一个字符串。
- **期望值绑死模型的探针会集体失效**：换运动模型后，11 个探针因为期望值是按旧模型写死的而全部变红，只能先停用再重写。

2.7 感想

这次考核让我第一次完整经历"从能跑，到可验证、可诊断、可交付"的差别。以前我对"代码写完了"的定义是"功能对了"，这次的定义变成了三条：**能不能用数据证明它对（对拍、探针、真编体积），出问题时能不能自己说出它错在哪（版本号、任务标签、停机上报），换个人拿到手能不能复**

现（文档、单文件交付稿、一键脚本）。前两条我以前基本不做，第三条我以前靠手动整理，这次全部脚本化，反而省了时间。

另一件体会是**实测优先于推理**。运动模型那件事上，我的推理链条是完整的，自检也是全绿的，错的是前提——只有把实物姿态摆出来逐条对，才可能发现"自洽但错"。后来我在容量、手感、限位这些事情上都尽量先测再改。

还有一点来自小组协作：机械、电路、物资和固件是四条并行的线，固件这边的每一次改动（工作空间窗口、限位、夹爪力度）都会影响硬件侧的假设，所以我把"改完先同步、写进文档再往下走"当成纪律；这次小组没有出现因为信息不同步而返工的情况，这一点我认为比多写几百行代码更值。

最后是关于节奏：16 天 38 次提交，平均每天两次以上。前期我常常一次提交改很多东西，出问题后很难定位；后期我把"一次提交只做一件事、提交信息写清为什么"当成纪律，回退和排查都轻松很多。这部分习惯我会继续保留。

附：可复核材料索引

- 固件与文档仓库：GitHub `2122019145/meArm`（38 次提交、18 个版本标签）
- 运动学对拍：`.selfcheck/kin_verify/verify_final.cpp`
- 串口链路回归：`.selfcheck/serialpick.cpp`；绘图工具：`.selfcheck/drawtrace.cpp`、`drawsweep.cpp`、`drawcheck.cpp`
- 严格编译与体积：`.selfcheck/compile_only.ps1`、`.selfcheck/avr_build.ps1`
- 交付生成脚本：`.selfcheck/make_single.ps1`、`make_serial_single.ps1`、`make_keypad_single.ps1`